Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Airport and Runway Management System

Airport and Runway Management System

Team Name: *Team #*	*THE VISIONARIES*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Padhalni,Subhi – 2510011725* *subhipadhalni@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Mongia,Pragya – 2510011608* *mongiapragya@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*–Baliyan,Taniya – 2510010174* *taniyabaliyan57@gmail.com* 
z	

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

The Airport Runway and Flight Management System is a C++ and Data Structures based project that makes airport flight and runway management simple and efficient. When many flights are handled manually scheduling clashes and decision problems arise. The Airport Runway and Flight Management System offers an effective solution that uses Data Structures and priority rules to fix these problems.

The Airport Runway and Flight Management System gathers data for each flight, such as flight number, flight name, arrival and departure time fuel amount and distance travelled. After collecting this data the Airport Runway and Flight Management System assigns a priority to each flight. Medical emergency flights receive the priority followed by flights with less fuel, VIP visits and normal flights. When two flights share the priority level the Airport Runway and Flight Management System compares their fuel amounts. The flight with fuel receives higher priority because it may need immediate attention. If fuel amounts are equal the Airport Runway and Flight Management System gives priority to the flight that has travelled a distance. If all factors are identical the Airport Runway and Flight Management System gives priority to the flight entered into the system.

The Airport Runway and Flight Management System uses a priority queue to arrange flights based on their priorities. Once the Airport Runway and Flight Management System has prioritized the flights it uses a queue to keep track of the order in which runways are allocated. The Airport Runway and Flight Management System manages two runways. Assigns flights one after another according to their priority levels. The Airport Runway and Flight Management System uses a stack to store completed flights demonstrating the Last-In-First-Out principle of Data Structures.

The Airport Runway and Flight Management System aims to reduce scheduling conflicts and improve runway allocation. The Airport Runway and Flight Management System shows how C++ and Data Structures can be used to build real-world airport management systems. The Airport Runway and Flight Management System serves as a foundation, for work including real-time scheduling, database storage, dynamic runway allocation and conflict detection.

Updated Project Approach and Architecture (2 pts)

This is a single-process console-based C++ application that implements a pipeline architecture. The flights' data are input from the console, followed by specific operations. Particularly, the user is requested to input the flight number, name, arrival and departure times, fuel, distance, and priority level (1 – 4, where 1 is a Medical Emergency, 2 – Less Fuel, 3 – VIP Visit, 4 – Normal). These data are saved in the struct Flight objects. All the flights are held in a priority queue with a custom ComparePriority comparator. The priority is determined based on the following criteria:

• priority level (with lower numbers indicating higher priority)
• less fuel
• greater distance
• entryOrder (implicitly, since it is used as a tie-breaker when flights have the same priority, fuel, and distance)

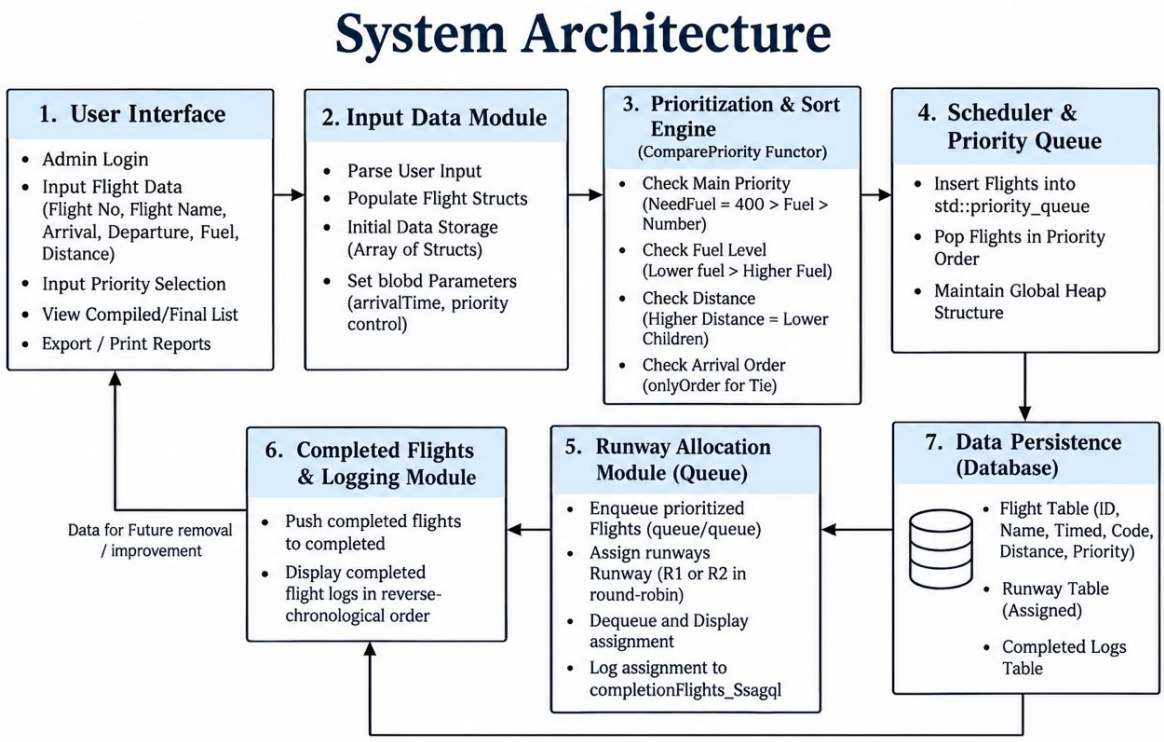
The flights are extracted from the priority queue and added to a queue (runway queue) in the order of highest priority.

Then, they are removed from the runway queue and added to the Runway 1 or Runway 2 in the round-robin way. The processed flights are then pushed to the stack to represent the completed flights list in the reversed order (last-in-first-out) manner. Communication protocols There are no network communication components in the application. The input and output streams are implemented as standard streams (stdin/stdout). The data exchange between the components is done using standard I/O streams and passing values via function calls.

Data structures

The following data structures are used:

• array – for storing the flights' data
• priority queue – for storing the flights in priority order
• queue – for storing the flights in the order of runway assignment
• stack – for storing the flights in the order of their completion graphicx



Tasks Completed (7 pts)

.

Task Completed	Team Member
1.Defined the Flight structure and its flight, timing, fuel, distance to priority, and entry-order fields. 2.Implemented the user input of flight details and priority choice. 3.Implemented the priority comparison using the priority category, fuel amount, distance, and entry order. 4.Used the priority queue to establish the flight allocation order. 5.Used a queue to process the runway assignment and the stack to display the completed flights. 6.Printed the flight details for each runway assignment and a completion summary.	MEMBER 1(1,2) MEMBER 2(3,6) MEMBER 3(4,5)

Challenges/Roadblocks (7 pts)

Challenges Faced and Planned Solutions
Challenges Faced and Roadblocks
In our project, Airport Runway and Flight Management System, we faced challenges in managing the flight details and managing the flights according to their priority. We also encountered some problems while taking user input for the flight name with spaces. Understanding the implementation of the various data structures such as arrays, stacks, queues, and priority queues was also a big challenge. We are currently working on enhancing the code, testing various test cases, and implementing the priority management of the flights. This process is still under validation.

Tasks Pending (7 pts)

Tasks	Team Member (to complete the task)
Fix the input-buffer issue to ensure flight names with spaces are entered correctly. Add input validation to prevent invalid flight numbers, fuel values, distances, and menu choices. Add a limit check to prevent the flight array from exceeding its capacity of 20 flights. Improve the priority queue comparator to handle emergency priority, fuel amount, and distance correctly. Improve runway allocation to avoid assigning overlapping flights to the same runway. Test the program with different inputs and correct any errors found during execution. Complete the final code review, project documentation, and presentation. All team member	MEMBER 1(1,2) MEMBER 2(3,6) MEMBER 3(4,5,7)

Project Outcome/Deliverables (2 pts)

The major outcome of the Airport Runway and Flight Management System project is a working C++ and Data Structures program that schedules the flights according to their priority and manages the allocation of the runways. The project consists of the following deliverables: a C++ program that handles the data entry and management of the flight details such as number/name, arrival/departure time, fuel percentage, and distance; a priority queue that sorts the flights according to the level of medical emergency, low fuel, VIP visit, and normal; a queue-based runway allocation that distributes the flights to the two available runways based on their priority; a stack-based completed flight record that showcases the LIFO policy; a complete working flight management system that provides the flight details, priority, and allocated runway; testing of the priority allocation and scheduling process; and documentation of the project, which includes abstract, methodology, individual efforts, challenges, results, and future scope. Overall, the project highlights the flight scheduling and runway allocation problem and develops a working solution based on C++ and Data Structures programming concepts. In addition, the project outcomes can be integrated into future studies, including the application of the project's results in the fields of management, engineering, and computer programming.

Progress Overview (2 pts)

The main program flow is implemented in the provided source code: data entry, priority assignment, priority-queue ordering, runway queue processing, and completed-flight stack display. The report and code structure can be prepared from the current implementation. Validation, systematic testing, evidence screenshots, and final documentation are still pending unless they have been completed separately. The code assigns runway numbers in alternating order; it does not model simultaneous runway availability or actual arrival-time scheduling.

Codebase Information (2 pts)

C++ Standard libraries used: iostream, string, queue, and
stack. The program uses Flight and ComparePriority structures.
priority$_{q}ueue, queue, andstackdatastructuresareimplemented.The$
repository link, the branch name, and the commit history were not provided
REPOSITORYLINK-https://github.com/pragyamongia2303/airport-runway-and-flight-management-system
by the user; add the details if the code is stored in any version-control repository.

Testing and Validation Status (2 pts)

Test Type	**Status (Pass/Fail)**	**Notes**
input testing	Pass	Flight details can be added into the system.
flight priority testing	pass	priority is assigned to emergency,low,fuel,VIP.
basic runway allocation	pass	flights are managed according to the priority.
equal priority testing	pending	cases where two flight have the same priority to be checked.
final system testing	pending	The complete system needs to be tested with different flight.

Deliverables Progress (2 pts)

The main part of our Airport Runway and Flight Management System has been completed. We have written the C++ code and added data structures like arrays, stacks, queues, and priority queues to manage flight details and runway allocation. The system also considers flight priorities, such as medical emergencies, low fuel, and VIP visits. We are currently working on testing the code, checking for errors, and making sure the system gives the correct output. Some testing and validation work is still pending.